Cluster RMI peer-to-peer per la custodia e classificazione distribuita di immagini

Relazione del progetto d'esame — Algoritmi Distribuiti

Emanuele Ugolotti
Unimore
Docente: Prof. Giacomo Cabri

24 ottobre 2026

Indice

1 Introduzione e contesto **2**

2 Requisiti (SRS) **2**
2.1 Attori e ambito . . . 2
2.2 Requisiti funzionali . . . 2
2.3 Requisiti non funzionali . . . 3
2.4 Assunzioni e vincoli . . . 3

3 Architettura **3**
3.1 Componenti . . . 3
3.2 Scelte progettuali . . . 4

4 Protocolli **5**
4.1 Sottomissione, classificazione e replica . . . 5
4.2 Rilevamento del guasto ed elezione (Bully) . . . 5
4.3 Avvio, rientro e sincronizzazione . . . 6

5 Implementazione e verifica **6**

6 Limiti e sviluppi futuri **7**

1 Introduzione e contesto

Il progetto nasce dall'esperienza di tirocinio svolta in Canada con il Prof. Peter Lewis sull'analisi delle macchie di sangue (*Bloodstain Pattern Analysis*, BPA) con tecniche di machine learning. Per altri corsi (IoT e Mobile; Cloud e Edge) è stato sviluppato un sistema composto da: un nodo di acquisizione e un gateway basati su ESP32 che comunicano via ESP-NOW; un backend Cloud (FastAPI, PostgreSQL, Docker) che classifica le immagini con una rete VGG16; un'applicazione Android che può classificare in locale e sincronizzarsi con il Cloud.

In quel sistema il Cloud è un **singolo punto di guasto**: se il backend o il database non sono raggiungibili, le immagini non sono consultabili né classificabili. Poiché in ambito forense le immagini sono potenziali prove, la loro disponibilità è un requisito rilevante.

Per il corso di Algoritmi Distribuiti è stato quindi aggiunto un componente a **oggetti distribuiti** basato su **Java RMI**: un *cluster peer-to-peer* di nodi che custodisce e classifica le immagini ricevute dal backend in modo replicato e tollerante ai guasti. Il cluster è indipendente dal resto del sistema, a cui si collega tramite un unico punto di contatto (il *bridge*, §3).

Nota sull'hardware. Il nodo di acquisizione ESP32 è stato restituito al docente del corso di IoT; le immagini arrivano quindi al backend dall'applicazione Android o da script. Questo non influisce sul componente descritto, che riceve immagini dal backend indipendentemente dalla loro origine.

2 Requisiti (SRS)

2.1 Attori e ambito

- **Backend Cloud**: sorgente delle immagini; le invia al cluster tramite il bridge.
- **Operatore**: interroga lo stato del cluster e recupera risultati (client a riga di comando o HTTP).
- **Nodo del cluster**: processo Java che offre e consuma servizi RMI (peer: sia client sia server).
- **Servizio di classificazione** (`robustness`): servizio esterno HTTP che esegue la VGG16.

2.2 Requisiti funzionali

ID	Requisito
FR1	Il sistema deve accettare un'immagine (identificativo + byte) da qualunque nodo del cluster.
FR2	Ogni immagine deve essere classificata da un solo nodo, il coordinatore, tramite il servizio di classificazione.
FR3	Il risultato (etichetta, confidenza, nodo che ha classificato, timestamp logico) deve essere replicato su tutti i nodi raggiungibili.
FR4	Un nodo non coordinatore che riceve una richiesta deve inoltrarla al coordinatore.
FR5	I nodi devono rilevare il guasto del coordinatore entro pochi secondi.
FR6	In caso di guasto del coordinatore il cluster deve eleggerne automaticamente uno nuovo, senza intervento umano.
FR7	Un nodo che si avvia o rientra deve recuperare i risultati che non possiede da un altro nodo.
FR8	Qualunque nodo vivo deve poter restituire un risultato già classificato, anche se il nodo che l'ha prodotto è caduto.

FR9	Il sistema deve esporre lo stato del cluster (nodi vivi, coordinatore, immagini possedute).
FR10	Il backend, non potendo usare RMI, deve poter inviare immagini e leggere lo stato tramite un ponte HTTP.

2.3 Requisiti non funzionali

ID	Requisito
NFR1	**Tolleranza ai guasti**: il sistema deve continuare a servire richieste con la caduta di qualsiasi singolo nodo, coordinatore incluso (modello *crash-stop*, con possibile rientro).
NFR2	**Disponibilità dei dati**: ogni risultato deve risiedere su più nodi.
NFR3	**Trasparenza**: il client non deve sapere quale nodo sia il coordinatore.
NFR4	**Non bloccanza**: l'indisponibilità del cluster non deve far fallire il flusso principale del backend.
NFR5	**Portabilità**: Java 17, solo libreria standard (nessuna dipendenza esterna).
NFR6	**Indipendenza**: il cluster deve funzionare anche senza bridge e senza backend.

2.4 Assunzioni e vincoli

Modello di sistema: processi che si fermano per guasto (crash-stop) e possono rientrare; canali affidabili ma con ritardi (i guasti si rilevano con *timeout*); sistema parzialmente sincrono. Non sono considerati guasti bizantini, partizioni di rete e perdita permanente di messaggi. I nodi hanno identificativi interi distinti e noti a priori (appartenenza statica).

3 Architettura

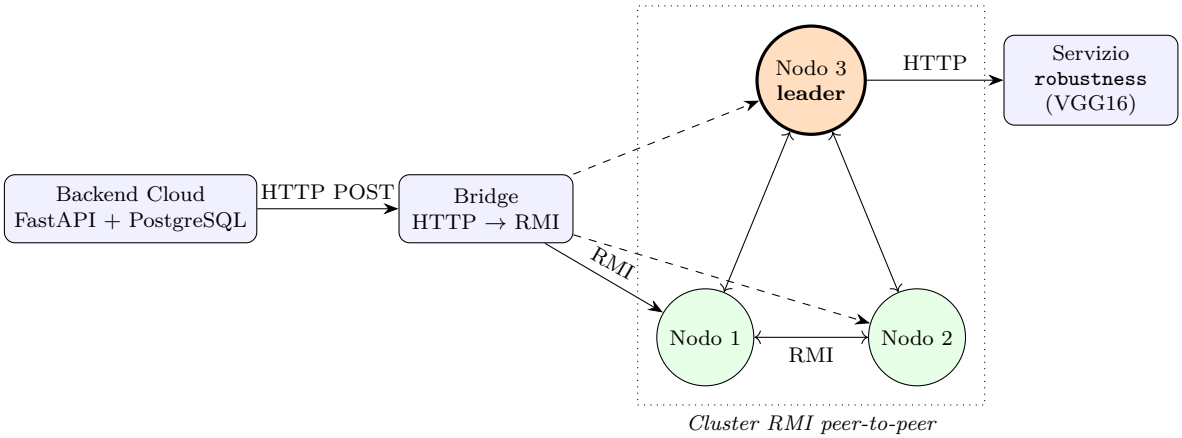


Figura 1: Architettura. Ogni nodo è sia client sia server RMI; il coordinatore (arancione) è l'unico che classifica. Il bridge contatta il primo nodo raggiungibile.

3.1 Componenti

`NodeService` (interfaccia remota)
Contratto RMI di ogni nodo (tabella 3).

`Node`
Implementa `NodeService` e la logica distribuita: heartbeat, elezione, replica, sincronizzazione, orologio di Lamport. Mantiene lo store locale dei risultati.

`ImageRecord`
Oggetto serializzabile che viene replicato: id, byte dell'immagine, etichetta, confidenza, nodo elaboratore, timestamp logico.

`Classifier`
Chiama via HTTP il servizio `robustness` (`/robustness/predict`) e ne estrae l'etichetta con probabilità massima. Se il servizio non è raggiungibile restituisce l'etichetta "non-classificata" senza interrompere il cluster.

`Bridge`
Server HTTP con tre endpoint (`/submit`, `/status`, `/get`) che traduce le richieste in chiamate RMI sul primo nodo raggiungibile. È necessario perché il backend è in Python e non può invocare RMI.

`Main`
Avvia un nodo (`Main <id> <n>`) o funge da client a riga di comando.

Metodo remoto	**Funzione**
`ping(from)`	Heartbeat: risponde `true` se il nodo è vivo.
`election(from)`	Bully: richiesta di elezione; il ricevente risponde e rilancia l'elezione.
`coordinator(id)`	Bully: annuncio del nuovo coordinatore.
`submit(id,data)`	Sottomette un'immagine; se il nodo non è leader inoltra al leader.
`replicate(rec)`	Il leader consegna un risultato da memorizzare.
`get(id)`, `listIds()`	Lettura di un risultato e dell'elenco locale (per la risincronizzazione).
`status()`	Stato leggibile del nodo.

Tabella 3: Interfaccia remota `NodeService`.

3.2 Scelte progettuali

Peer-to-peer e non client-server. Ogni nodo espone gli stessi servizi e può ricevere richieste; non esiste un server fisso. Un ruolo (il coordinatore) esiste, ma è dinamico ed elettivo: è una funzione assunta da un peer, non un'entità distinta.

Deployment. Ogni nodo ha un proprio registro RMI sulla porta $1100 + id$ e l'oggetto esportato sulla porta fissa $1200 + id$, così da poter eseguire più nodi sulla stessa macchina e, in prospettiva, su macchine diverse. Nella dimostrazione i nodi sono processi distinti sullo stesso computer.

Timeout RMI. Sono impostati a 800 ms; senza di essi la chiamata a un nodo caduto potrebbe bloccarsi a lungo e il guasto non verrebbe rilevato in tempi utili.

4 Protocolli

Si descrivono i protocolli con diagrammi di sequenza. Le linee tratteggiate sono risposte; i riquadri gialli sono azioni locali.

4.1 Sottomissione, classificazione e replica

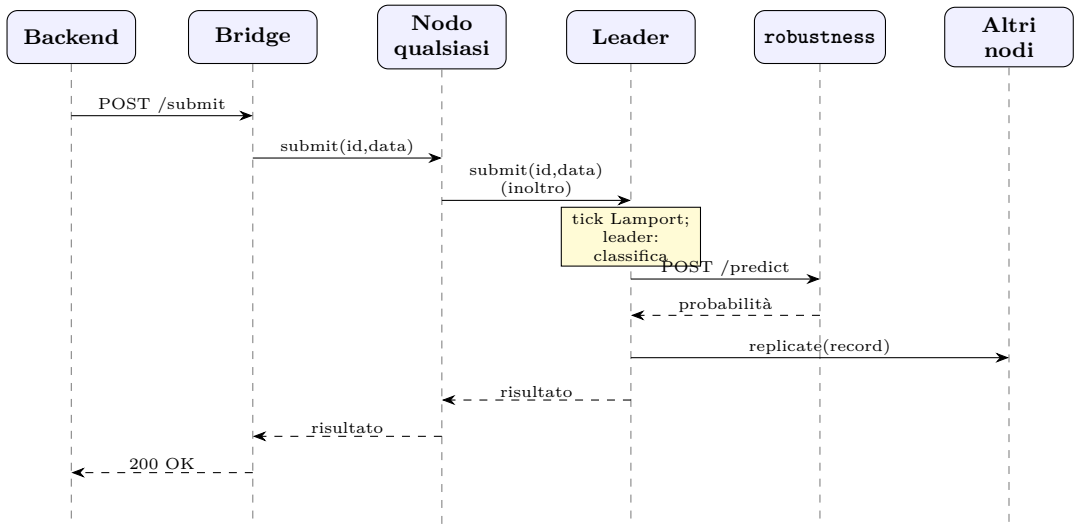


Figura 2: Flusso di una richiesta. Il nodo contattato non deve essere il leader (trasparenza, NFR3).

Il leader incrementa l'orologio di Lamport a ogni sottomissione e assegna il valore al record. Ogni ricevente aggiorna il proprio orologio con $L \leftarrow \max(L, L_{rec}) + 1$. La replica è *best effort*: se un nodo non risponde, l'errore è registrato e il risultato resta comunque su tutti gli altri; il nodo mancante lo recupererà con la sincronizzazione (§4.3).

4.2 Rilevamento del guasto ed elezione (Bully)

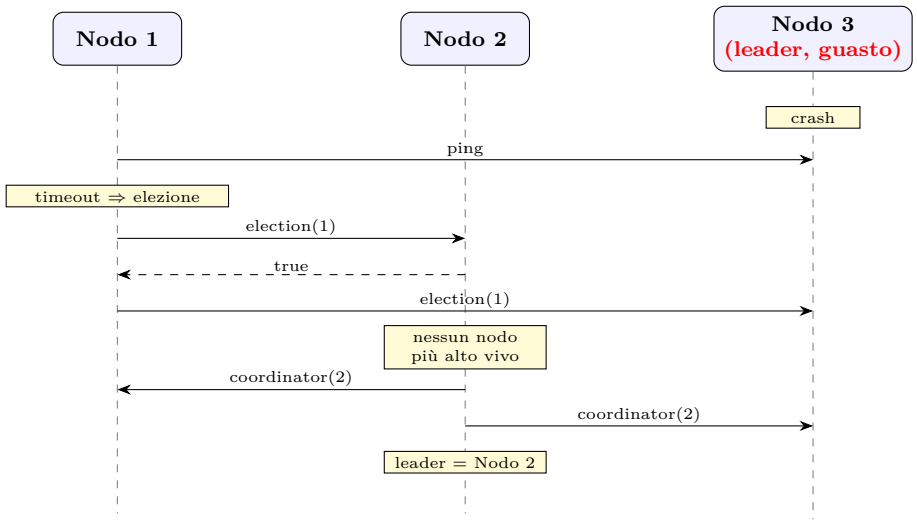


Figura 3: Il nodo 1 rileva l'assenza del leader, avvia l'elezione e il nodo 2 (il più alto vivo) si proclama coordinatore. Le richieste al nodo 3 cadono per timeout e sono ignorate.

Algoritmo. Un nodo che rileva l'assenza del coordinatore invia `election` a tutti i nodi con identificativo più alto. Se nessuno risponde, si proclama coordinatore e lo annuncia a tutti. Se almeno uno risponde, quello rilancia l'elezione a sua volta; se l'annuncio non arriva entro 3 s l'elezione viene ritentata. Vince così il nodo vivo con identificativo massimo.

Rilevamento. I nodi non coordinatori inviano un `ping` al coordinatore ogni secondo; la mancata risposta (o il timeout di 800 ms) fa partire l'elezione. Il rilevatore è quindi basato su timeout, *non perfetto*: in rete lenta potrebbe segnalare falsi guasti (§6).

4.3 Avvio, rientro e sincronizzazione

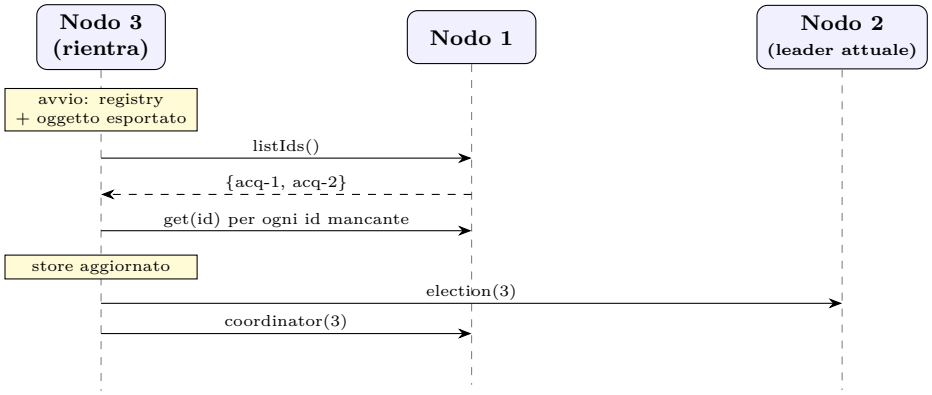


Figura 4: Un nodo che rientra recupera i dati mancanti e, avendo l'identificativo più alto, riassume il ruolo di coordinatore (comportamento tipico di Bully).

5 Implementazione e verifica

Tecnologie. Java 17, RMI, `java.net.http` per il client HTTP, `com.sun.net.httpserver` per il bridge. Nessuna libreria esterna.

Avvio. Si avviano i nodi (`java -cp out bpa.Main <id> <n>`), poi il bridge (`java -cp out bpa.Bridge <n> 9000`). Il backend inoltra le immagini al bridge; il cluster può essere disattivato lasciando vuota la variabile `RMI_BRIDGE_URL`.

Scenario di prova. Con 3 nodi e il bridge:

1. invio di un'immagine al bridge: classificata dal leader (nodo 3), replicata su tutti i nodi;
2. arresto del leader: entro pochi secondi il nodo 2 diventa coordinatore, `status` mostra il nodo 3 non raggiungibile;
3. nuova immagine: elaborata dal nodo 2; la prima immagine resta leggibile da tutti i nodi vivi (FR8);
4. riavvio del nodo 3: recupera le immagini e riassume la leadership.

La prova automatica è stata eseguita con un servizio di classificazione simulato che restituisce risposte nel formato del servizio reale; l'integrazione con il servizio `robustness` effettivo è verificabile con la procedura descritta nel README del codice.

6 Limiti e sviluppi futuri

- **Partizioni di rete**: con una partizione possono coesistere due coordinatori (*split brain*); non c'è riconciliazione al ricongiungimento.
- **Consenso**: la replica è guidata dal leader e non è un protocollo di consenso; una scrittura può essere persa se il leader cade prima di replicarla.
- **Rilevatore di guasti**: solo i follower controllano il leader (nessun controllo tra follower) e i timeout possono generare falsi positivi.
- **Orologio di Lamport**: ordina gli eventi di replica ma non risolve conflitti tra scritture concorrenti.
- **Appartenenza statica**: l'elenco dei nodi è fissato all'avvio.
- **Persistenza**: lo store è in memoria; un riavvio di tutti i nodi azzera i dati.
- **Distribuzione fisica**: nella dimostrazione i nodi girano sullo stesso computer; il codice è predisposto per macchine diverse ma non è stato provato in tale configurazione.

Sviluppi. Protocollo di consenso (Raft) al posto della replica semplice; appartenenza dinamica; persistenza su disco; deployment su più macchine; ordinamento totale degli eventi con clock vettoriali.